

HUMORIX AI

Blueprint v3.0

Hardened Edition

Memory Layer - Dynamic Fusion - Uncertainty Scoring - Edge Privacy - Intervention Validation - Personalization - Resilience Protocol

Version	3.0 - Hardened Edition (builds on v2.0 Developer Blueprint)
What's New	Dynamic fusion weights · Uncertainty scoring · Semantic crisis detection · Affective memory (vector store) · On-device ONNX inference · Intervention efficacy loop · Personalization layer · Control-mode toggle · Visible privacy UX · Evaluation metrics framework
Disclaimer	Supportive tool only. Not a substitute for licensed professional mental health care.

NEW v3.0 is a direct response to critical review of v2.0. Every change is tagged [UPGRADE] or [NEW] throughout this document so you can track exactly what changed and why.

What Changed from v2.0 to v3.0

9 critical improvements — tagged throughout the document

v2.0 BLUEPRINT	v3.0 HARDENED
Static fusion weights (0.45 / 0.30 / 0.25)	Dynamic weights — degrade gracefully when modality unreliable
No uncertainty output	overall_uncertainty score added — softer responses when AI is unsure
Keyword-only crisis detection	Semantic intent classifier + multi-message pattern detection
Single-moment conflict handling	Session-level denial pattern tracking across multiple messages
Generic DVITE for all users	Personalization layer: response style, length, tone memory
Satvic always-on	User preference gate — Satvic suppressed if not opted-in
UI with no explicit control mode	Control Mode Toggle: Minimal / Voice-only / Full Multimodal
Privacy as policy only	Visible real-time processing indicators + deletion confirmation UX
No success measurement	Full evaluation metrics: efficacy loop, KPIs, clinical dashboard

Table of Contents

A — System Architecture (Unchanged Foundation)

- A1 Five-Layer Pipeline
- A2 File Structure v3
- A3 WebSocket Flow

C — Dynamic Fusion Engine v3

- C1 Dynamic Weight Calculation (UPGRADE)
- C2 Uncertainty Score (NEW)
- C3 Degradation Protocol (NEW)

E — DVITE + Personalization

- E1 Protocol Overview
- E2 Personalization Layer (NEW)
- E3 Disorder Module Library
- E4 Intervention Efficacy Loop (NEW)

G — Satvic Wellness — Repositioned

- G1 User Preference Gate (UPGRADE)
- G2 Wellness Nudge retained
- G3 Meal Plans & Food Laws

I — Edge Privacy & Deployment

- I1 ONNX On-Device Inference (UPGRADE)
- I2 Ephemeral Processing retained
- I3 Deployment Modes

K — Ethics, Limitations & Roadmap

- K1 Non-Negotiables
- K2 Known Gaps
- K3 v4.0 Roadmap

B — Safety Middleware — Hardened

- B1 Semantic Crisis Detection (UPGRADE)
- B2 Multi-Message Pattern Tracking (NEW)
- B3 Existing Keyword Layer retained

D — Affective Memory Layer

- D1 Vector Store Architecture (NEW)
- D2 Privacy: Local Encrypted Store
- D3 Longitudinal Context Usage

F — Conflict Handling — Upgraded

- F1 Session-Level Denial Tracking (UPGRADE)
- F2 Empathetic Inquiry Protocol retained

H — UI System — Hardened

- H1 Control Mode Toggle (NEW)
- H2 Visible Privacy UX (NEW)
- H3 Adaptive States retained

J — Evaluation Metrics & KPIs

- J1 Intervention Efficacy Loop (NEW)
- J2 System KPIs (NEW)
- J3 Clinical Dashboard

A

System Architecture

Foundation unchanged · New modules added at Decision and Data layers

A1 Five-Layer Pipeline — v3 Updates

#	Layer	v2 Components	v3 Additions
1	INPUT	Webcam · Mic · Text	Control Mode toggle gates which inputs are active
2	PROCESSING	OpenCV · Librosa · Tokenizers	Confidence scoring per frame now feeds dynamic weight calculator
3	EMOTION	FER · Voice ML · Sentiment	Uncertainty score computed alongside each modality output
4	DECISION	Fusion · DVITE · Satvic KB	+ Affective Memory (ChromaDB) · Personalization Layer · Efficacy Loop
5	OUTPUT	FastAPI · Streamlit · Crisis	+ Real-time processing indicator · Deletion confirmation UX

A2 File Structure — v3

```
humorix/  
  app/  
    main.py  
  core/  
    safety.py # Gatekeeper – keyword + semantic (UPGRADE)  
    fusion.py # Dynamic weights + uncertainty score (UPGRADE)  
    triage.py # DVITE + personalization layer (UPGRADE)  
    memory.py # Affective vector store (NEW)  
    efficacy.py # Intervention success measurement (NEW)  
    wellness.py # Satvic – preference-gated (UPGRADE)  
  models/  
    vision/ # MediaPipe + DeepFace (unchanged)  
    audio/ # Librosa + speech model (unchanged)  
    nlp/ # Transformer sentiment (unchanged)  
    intent/ # Semantic crisis intent classifier (NEW)  
  data/  
    schemas.py  
    protocols.py  
    resources.py  
    user_profile.py # Preference + style memory (NEW)  
    vector_store/ # Local encrypted ChromaDB (NEW)  
    affective_context.db
```

B

Safety Middleware — Hardened

Keyword layer retained · Semantic detection added · Multi-message patterns

!! The v2 keyword list was a necessary start — but people in crisis rarely use textbook language. 'I'm tired of everything' carries the same risk as 'I want to die.' v3 adds two layers above keywords.

B1 Semantic Crisis Detection [UPGRADE]

Intent classifier runs in parallel with keyword check

A fine-tuned transformer intent classifier runs on every input alongside the keyword check. It detects semantic hopelessness and veiled crisis intent even when no trigger word is present.

```
# app/core/safety.py (v3)

CRISIS_KEYWORDS = {'kill', 'die', 'suicide', 'end it', 'hurt myself', 'cant go on'}

SEMANTIC_RISK_PHRASES = [
    'tired of everything', 'nothing matters anymore', 'wish I could disappear',
    'everyone would be better off', 'no point anymore', 'cant do this anymore'
]

def run_safety_check(text: str, intent_model) -> dict:
    # Layer 1: Fast keyword scan (< 2ms)
    if any(kw in text.lower() for kw in CRISIS_KEYWORDS):
        return _crisis_response('keyword')

    # Layer 2: Semantic phrase patterns (< 5ms)
    if any(phrase in text.lower() for phrase in SEMANTIC_RISK_PHRASES):
        return _crisis_response('semantic_phrase')

    # Layer 3: Intent classifier (< 20ms, runs async)
    intent_score = intent_model.predict(text) # Fine-tuned on crisis datasets
    if intent_score['crisis_probability'] > 0.75:
        return _crisis_response('intent_classifier')

    return {'status': 'SAFE', 'action': 'PROCEED'}
```

B2 Multi-Message Pattern Tracking [NEW]

Emotions are patterns, not snapshots

A session-level buffer stores the last N messages. If hopelessness indicators appear across multiple messages — even if none individually cross the threshold — the system escalates its sensitivity and prompts an empathetic check-in.

```
# app/core/safety.py — pattern tracker

class SessionSafetyTracker:
    def __init__(self, window=5):
        self.buffer = [] # Last N message risk scores
```

```
self.window = window

def update(self, risk_score: float):
    self.buffer.append(risk_score)
    if len(self.buffer) > self.window:
        self.buffer.pop(0)

def sustained_risk(self, threshold=0.4) -> bool:
    # True if average risk across window exceeds threshold
    if len(self.buffer) < 3:
        return False
    return sum(self.buffer) / len(self.buffer) > threshold
```

C

Dynamic Fusion Engine v3

Adaptive weights · Uncertainty score · Degradation protocol

! v2 used fixed weights (0.45 / 0.30 / 0.25). The core problem: bad lighting or background noise made those weights actively misleading. v3 asks 'who is trustworthy right now?' and leads with that.

C1 Dynamic Weight Calculation [UPGRADE]

```
# app/core/fusion.py (v3)

BASE = {'face': 0.45, 'voice': 0.30, 'text': 0.25}

def dynamic_weights(face_conf: float, voice_conf: float,
env_noise_high: bool = False) -> dict:
    w = BASE.copy()

    # Degrade face weight in unreliable conditions
    if face_conf < 0.5:
        deficit = w['face'] * (1 - face_conf / 0.5)
        w['face'] -= deficit
        w['voice'] += deficit * 0.6 # Voice picks up most of face's slack
        w['text'] += deficit * 0.4

    # Degrade voice weight in noisy environments
    if env_noise_high and voice_conf < 0.5:
        deficit = w['voice'] * 0.5
        w['voice'] -= deficit
        w['text'] += deficit

    # Normalise to sum = 1.0
    total = sum(w.values())
    return {k: v / total for k, v in w.items()}

def calculate_fusion(face: float, voice: float, text: float,
face_conf: float, voice_conf: float,
env_noise: bool = False) -> dict:
    w = dynamic_weights(face_conf, voice_conf, env_noise)
    fused = face*w['face'] + voice*w['voice'] + text*w['text']
    if max(face, voice, text) > 0.9:
        fused = max(face, voice, text) # Trust a high-confidence signal
    return {'fused_score': fused, 'weights_used': w}
```

C2 Uncertainty Score [NEW]

How sure is the AI? Softer responses when it isn't.

The uncertainty score is the system's humility metric. When overall_confidence is low, the AI adapts its language — hedging rather than asserting, asking rather than prescribing. This prevents the system from sounding like a fake therapist that claims certainty it doesn't have.

```
# Computed after every fusion call

def compute_uncertainty(overall_confidence: float,
weight_variance: float) -> dict:
    uncertainty = 1.0 - overall_confidence
    return {
        'overall_uncertainty': round(uncertainty, 3),
        'response_mode': 'tentative' if uncertainty > 0.4 else 'direct'
    }

# Response tone examples based on uncertainty:
# uncertainty > 0.4 -> 'I'm noticing something that might be anxiety – does that feel right?'
# uncertainty < 0.2 -> 'I can see you're experiencing significant anxiety right now.'
```

C3 Degradation Protocol [NEW]

Formal fallback logic — never fail silently

Condition	Trigger	System Response	UI Message
Face unavailable	face_conf < 0.3	Face weight set to 0. Voice + text carry 100%.	'Video analysis unavailable — switching to voice support'
Voice noisy	env_noise_high AND voice_conf < 0.4	Voice weight halved. Text compensates.	'Background noise detected — relying on text signals'
All low-confidence	All modalities < 0.35	Soft mode: no intervention pushed. Ask user directly.	'I want to make sure I understand — how are you feeling right now?'
Text only mode	User selects text-only in Control Mode	Face weight = 0, voice weight = 0, text weight = 1.0	No camera/mic indicator shown

D

Affective Memory Layer

Vector store · Local encrypted · Longitudinal context · Privacy-first

NEW If a user mentions a conflict at work on Tuesday, the AI shouldn't have forgotten it by Friday. The Memory Layer stores contextual emotional embeddings — not raw biometrics — locally on the device.

D1 Vector Store Architecture [NEW]

Database	ChromaDB (local) — lightweight, Python-native, no server required
What is stored	Semantic embeddings of: session emotional themes · trigger contexts · successful intervention types · disorder risk history
What is NOT stored	Raw video frames · audio recordings · biometric readings — never persisted
Embedding model	sentence-transformers (e.g. all-MiniLM-L6-v2) — runs entirely on-device
Encryption	AES-256 encryption on the ChromaDB file. Key stored in device keychain.
Retention	Default: 90 days. User-configurable. Full deletion on request (immediate).

```
# app/core/memory.py

import chromadb
from sentence_transformers import SentenceTransformer

class AffectiveMemory:
    def __init__(self):
        self.client = chromadb.PersistentClient(path='./vector_store') # Local only
        self.col = self.client.get_or_create_collection('affective_context')
        self.model = SentenceTransformer('all-MiniLM-L6-v2')

    def store_session(self, session_summary: str, metadata: dict):
        embedding = self.model.encode(session_summary).tolist()
        self.col.add(embeddings=[embedding],
            documents=[session_summary],
            metadatas=[metadata],
            ids=[metadata['session_id']])

    def retrieve_context(self, current_input: str, n=3) -> list:
        query_emb = self.model.encode(current_input).tolist()
        results = self.col.query(query_embeddings=[query_emb], n_results=n)
        return results['documents'][0] # Top-N relevant past contexts
```

D2 Longitudinal Context Usage

Use Case	How Memory Is Used	Clinical Value
Repeated triggers	Detect that 'work stress' appears in 3 of last 5 sessions	Flag as persistent stressor — suggest dedicated GAD module

Intervention history	Track which techniques worked vs. which were skipped	Stop suggesting interventions the user consistently dismisses
Denial pattern	Log 'user said OK / biometrics said distress' mismatches	After 5+ mismatches, gently probe for disclosure readiness
Progress tracking	Compare severity scores across sessions	Visualise trend in session history sidebar
Satvic alignment	Track which wellness suggestions led to positive follow-up mood	Personalise wellness recommendations over time

E

DVITE + Personalization Layer

Protocol unchanged · Personalization added · Efficacy loop new

E1 DVITE Protocol — Unchanged Core

01 DETECT Fusion outputs emotion + severity score	02 VALIDATE Clinically-worded empathy response	03 INTERVENE CBT/DBT technique matched to severity	04 TRACK User rates 1-10 Trend computed	05 ESCALATE Threshold breach -> helplines
---	--	--	---	---

E2 Personalization Layer [NEW]

Same protocol — different voice, pace, and depth per user

DVITE was clean but generic — same tone for every user, every session. The personalization layer wraps the protocol: before generating any response, it reads the user's style profile and adjusts accordingly. No structural change to DVITE itself.

Dimension	Options	How Detected / Set
Response length	Short / Medium / Long	User preference at onboarding + adaptive from engagement patterns
Tone	Warm-emotional / Analytical-logical / Balanced	Detected from user's own language patterns across first 3 sessions
Intervention style	Sensory-first (grounding) / Cognitive-first (CBT) / Mixed	Tracked from which interventions the user completes vs. skips
Fatigue detection	Flag if user has received same technique 3x in session	Auto-rotate to next technique in same tier
Disclosure speed	Fast discloser / slow discloser	Inferred from how many sessions before significant personal sharing

```
# data/user_profile.py

class UserProfile:
    response_length: str = 'medium' # short | medium | long
    tone: str = 'balanced' # warm | analytical | balanced
    preferred_modality: str = 'mixed' # sensory | cognitive | mixed
    fatigue_log: list = [] # last N interventions used
    denial_count: int = 0 # 'OK' + high-bio mismatches
    disclosure_speed: str = 'unknown' # fast | slow | unknown
```

E3 Intervention Efficacy Loop [NEW]

Measures whether the intervention actually worked

The current system selects an intervention and moves on. The efficacy loop adds a measurement step: after each intervention, the system re-scans biometrics for 60 seconds and checks whether distress indicators have reduced. This turns a static tool into an adaptive clinical assistant.

```
# app/core/efficacy.py

def measure_efficacy(pre_score: float, post_score: float,
intervention_type: str, user_id: str) -> dict:
    change = pre_score - post_score # Positive = improvement
    pct = (change / pre_score) * 100 if pre_score > 0 else 0

    if pct >= 20:
        result = 'effective'
    elif pct >= 5:
        result = 'partial'
    else:
        result = 'ineffective'

    # Log to user profile for personalization
    log_intervention_result(user_id, intervention_type, result)

    # If ineffective: try next intervention in same tier
    # If still ineffective after 2 attempts: escalate severity level
    return {'result': result, 'pct_change': round(pct, 1)}
```

Result	Criterion	System Action
Effective	> 20% bio score reduction	Affirm, log success, continue
Partial	5-20% reduction	Continue, offer one additional technique
Ineffective	< 5% reduction	Try next technique in tier; after 2 fails — escalate severity

E4 Disorder Module Library — 8 Conditions (unchanged)

Code	Condition	Escalate	Techniques (by severity)
GAD	Generalised Anxiety	7/10+	Mild: 5-4-3-2-1 · Extended exhale Mod: 4-7-8 · PMR · Thought challenge Severe: CBT referral
PANIC	Panic Disorder	8/10+	Mild: Box breathing · Floor grounding Mod: DARE Severe: Crisis stabilisation
MDD	Major Depressive Disorder	7/10+	Mild: Activation · Gratitude Mod: Pleasure prediction · Thought record Severe: Safety inquiry
PTSD	Post-Traumatic Stress	7/10+	Mild: Grounding anchor · Safe place Mod: Window of tolerance Severe: EMDR referral
MANIA	Bipolar — Manic Episode	7/10+	Mild: Sleep regulation Mod: 48-hr decision rule Severe: Psychiatrist contact
OCD	Obsessive-Compulsive	8/10+	Mild: Thought defusion · Delay Mod: ERP ladder Severe: Specialist referral
ED	Eating Disorders	6/10+	Mild: Body neutrality Mod: Voice challenging Severe: NAED helpline
ADHD	Attention Deficit	8/10+	Mild: 2-min rule · Pomodoro Mod: Task hacks Severe: Psychiatry eval

F

Conflict Handling — Upgraded

Session-level denial tracking added to v2 single-moment logic

F1 Session-Level Denial Pattern Tracking [UPGRADE]

v2 handled the conflict in a single moment. But if a user says 'I'm fine' while biometrics consistently show distress — across 5, 7, 10 messages — the system needs to recognise the pattern and gradually adapt its tone rather than repeating the same empathetic inquiry each time.

```
# Integrated into UserProfile

def handle_conflict(user_text: str, bio_score: float,
profile: UserProfile) -> dict:
    sentiment = analyze_sentiment(user_text)
    conflict = bio_score > 0.7 and sentiment < 0.5

    if conflict:
        profile.denial_count += 1

        if profile.denial_count <= 3:
            # Early stage: gentle empathetic inquiry
            tone = 'empathetic_inquiry'
            msg = 'I hear you are okay – I am noticing some tension. I am here.'

        elif profile.denial_count <= 7:
            # Mid stage: acknowledge the pattern, give more space
            tone = 'spacious_presence'
            msg = 'No pressure at all – I am just here whenever you are ready.'

        else:
            # Late stage: direct but gentle – name the mismatch
            tone = 'named_mismatch'
            msg = 'I have noticed a pattern and want to check in directly –'
            ' how are things actually going?'

    return {'tone': tone, 'response': msg, 'denial_count': profile.denial_count}

profile.denial_count = 0 # Reset on genuine self-disclosure
return {'tone': 'standard', 'response': 'Great to hear.'}
```

G

Satvic Wellness — Repositioned

Optional lifestyle layer · User preference gate · Nudge method retained

! The v2 risk: users came for mental health support and received diet advice. v3 makes Satvic explicitly opt-in and positions it as a lifestyle companion, never a therapeutic solution.

G1 User Preference Gate [UPGRADE]

Onboarding Question	'Would you like lifestyle and wellness suggestions (Satvic philosophy)? Yes / No / Ask me later'
Default	OFF — user must opt in. Not assumed.
Suppression Rule	Even if opted-in: suppressed when severity_score > 0.6 (unchanged from v2)
Preference Code	if user_profile.satvic_enabled == False: skip wellness module entirely
Positioning	Framed as: 'Optional lifestyle layer, not a therapeutic solution'

G2 Wellness Nudge — Retained from v2

OK The Wellness Nudge delivery method is unchanged. Contextual, soft, woven into conversation — not prescribed.

G3 Satvic Reference — 9 Laws & Meal Plans

#	Food Law	Avoid	Eat
1	No Dead Foods	Packaged / tinned	Fresh fruits, veg, grains — farm to kitchen
2	No Refined	White sugar, flour, refined oil	Dates, jaggery, millets, cold-pressed oils
3	No Animal	Meat, dairy, eggs	Coconut milk, almond milk, cashew (homemade)
4	Less Grain, More Veg	Grain-dominant plates	2x more vegetables than grain always
5	Seasonal & Local	Imported exotics	Regional seasonal produce
6	Soak Nuts	Unsoaked nuts	Soak 6-8h, discard water
7	Mild Spices	Packaged masalas	Fresh green chili, black pepper, herbs
8	Minimal Cooking	High-heat frying	Lowest temp, shortest time — raw if possible
9	No Stimulants	Tea, coffee, onions, garlic	Herbal teas, coconut water

H

UI System — Hardened

Control Mode Toggle new · Visible privacy UX new · Adaptive states retained

H1 Control Mode Toggle [NEW]

Users control exactly what the AI can see

Not every user wants the AI watching their face and analysing their voice at all times. The Control Mode toggle gives users explicit agency over which modalities are active — without reducing the quality of support in any mode.

Mode	Active Inputs	Fusion Weights	Use Case
Minimal (Text Only)	Text input only	Text 1.0 / Face 0.0 / Voice 0.0	Maximum privacy · public spaces · low-bandwidth
Voice + Text	Mic + Text	Voice 0.55 / Text 0.45 / Face 0.0	Private voice support without camera
Full Multimodal	Webcam + Mic + Text	Dynamic (face 0.45 base)	Maximum accuracy — full support capability

H2 Visible Privacy UX [NEW]

Trust is not just policy — it is UX

Users don't trust what they can't see. v3 adds real-time visible indicators that show exactly what is being processed and confirm when data is deleted — not buried in a settings menu.

Element	Behaviour	Location
Camera indicator	'Camera ON · Analysing' / 'Camera OFF' — colour-coded dot	Header bar — always visible
Mic indicator	'Voice ON · Processing' / 'Voice OFF'	Header bar — always visible
Processing badge	Subtle pulsing indicator while biometrics are being analysed	Bottom of Adaptive Canvas
Deletion confirmation	Animation: shield icon + 'Session data deleted' on confirm	Full-screen overlay for 2 seconds
Data age label	'Memory: 3 sessions stored · Last: 2 days ago'	Settings / privacy panel

H3 Adaptive Screen States — Retained from v2

State	Trigger	UI Behaviour
Check-in	Baseline / neutral	Standard gradient · normal pace · full interface
Grounding	Anxiety / panic detected	Muted blue-grey · slow text · breathing animation
Wellness	Low severity + Satvic opt-in	Nature palette · recipe cards · mindful prompts
Crisis	Crisis protocol triggered	Minimal UI · large text · helplines immediately visible

I

Edge Privacy & Deployment

On-device ONNX inference · Ephemeral processing · Deployment modes

I1 ONNX On-Device Inference [UPGRADE]

Data that never leaves the device cannot be breached

Converting PyTorch models to ONNX format enables the Humorix inference pipeline to run entirely on the user's device — no facial features, audio patterns, or text ever transmitted to a cloud server. This is the strongest possible privacy guarantee, and a prerequisite for NIMHANS clinical validation.

```
# Convert trained PyTorch model to ONNX (run once after training)
import torch, onnx

model = load_trained_emotion_model() # Your PyTorch model
dummy = torch.randn(1, 3, 224, 224) # Example input shape

torch.onnx.export(
    model, dummy,
    'models/emotion_model.onnx',
    input_names=['face_frame'],
    output_names=['emotion_probs'],
    dynamic_axes={'face_frame': {0: 'batch'}}
)

# Runtime inference on device — no network call
import onnxruntime as ort

session = ort.InferenceSession('models/emotion_model.onnx',
    providers=['CPUExecutionProvider'])

result = session.run(None, {'face_frame': frame_array})
```

I2 Deployment Modes

Mode	Stack	Privacy Level	When to Use
Local Dev	Python + Streamlit · CPU/GPU	High	Dev + personal use
ONNX Edge	ONNX Runtime · on-device	Maximum	Privacy-first production start
GPU Cloud	Docker + Triton · AWS/GCP	Medium	Scale — add consent + encryption
Mobile ONNX	ONNX + React Native / Flutter	Maximum	Consumer app — offline capable
Clinical API	FastAPI REST · JWT · TLS	High	EHR/telehealth integration

J

Evaluation Metrics & KPIs

Intervention efficacy · System KPIs · Clinical dashboard

NEW v2 had no success measurement. If you can't measure it, you can't improve it — and you certainly can't take it to NIMHANS for clinical validation.

J1 System-Level KPIs [NEW]

Metric	Definition	Target	Measurement Method
Intervention success rate	% of interventions where bio score drops > 20% post-exercise	> 60%	Efficacy loop (app/core/efficacy.py)
Crisis detection accuracy	% of genuine crises correctly identified	> 95%	Annotated test set + live flag review
False positive rate	% of non-crisis inputs flagged as crisis	< 3%	Annotated test set
Session completion rate	% of users who complete full DVITE loop	> 70%	Session tracking
Dropout rate	% who disengage after < 2 DVITE steps	< 20%	Funnel analytics
Modality reliability	% of sessions where face confidence > 0.5	Track only	Per-session logging
Uncertainty mean	Average overall_uncertainty score across sessions	< 0.35	Fusion engine logs

J2 Clinical Dashboard [NEW]

For professional review — not user-facing

Panel	Data Shown	Purpose
Trend Overview	Severity score across sessions (line chart)	Identify improving / stable / deteriorating users
Intervention Log	Which techniques used · efficacy result per attempt	Which techniques work for this user vs. population
Conflict Flags	Sessions with high denial_count	Flag users who may need professional outreach
Crisis Events	All crisis triggers with timestamp and resource shown	Audit trail for clinical compliance
Uncertainty Heatmap	Which sessions had high overall_uncertainty	Identify hardware or context reliability issues

K

Ethics, Limitations & v4 Roadmap

Non-negotiables unchanged · Gaps addressed · Next horizon

K1 Ethical Non-Negotiables — Unchanged

Principle	Status
No diagnosis — ever	Non-negotiable. Cannot be overridden.
Disclaimer every session	Non-negotiable. Cannot be disabled.
Crisis resources unconditional	Non-negotiable. Helplines foregrounded immediately.
Biometric consent before capture	Non-negotiable. Modal required.
Transparency of detection	Non-negotiable. System explains what it detected.
No third-party data sale	Non-negotiable.
Bias audit per release	Required. Covers skin tone, age, gender, culture.

K2 v3 Gaps — Honest Assessment

Gap	Why It Remains	v4 Plan
Demographic FER accuracy	Training data still skewed — takes time	South Asian + East African datasets in collection
No published clinical RCT	Requires partner institution and ethics approval	NIMHANS MOU in progress
Intent classifier training	Requires crisis-labelled dataset (sensitive)	Partner with crisis org for annotated data
ONNX mobile latency	Some models too heavy for older devices	Knowledge distillation to create lightweight variants
Indian language NLP	Transformer fine-tuning requires data + time	Hindi + Tamil models target for v4

K3 v4.0 Roadmap

WEARABLES	FEDERATED LEARNING	MULTILINGUAL NLP	CLINICAL RCT	LIGHTWEIGHT ONNX	VOICE CLONING DETECT
Initiative	Description				Priority
Wearable Integration	HRV, GSR, sleep data as 4th modality				High
Federated Learning	Per-user model fine-tuning without central data transmission				High
Indian Language NLP	Hindi, Tamil, Telugu, Bengali, Marathi				High
Clinical RCT	NIMHANS-partnered controlled trial				High
Lightweight ONNX Models	Knowledge distillation for mobile-first deployment				Medium

Voice Spoofing Detection	Detect synthetic voice input to prevent system gaming	Medium
Therapist Handoff API	Structured session summary export to licensed practitioner	Medium

Blueprint Version Summary

Area	v1 (Research)	v2 (Developer Blueprint)	v3 (Hardened)
Fusion	Fixed concept	Static 0.45/0.30/0.25	Dynamic + uncertainty score
Safety	Keyword list	Keyword list	Keyword + semantic + pattern
Memory	None	None	Local vector store (ChromaDB)
Privacy	Policy only	Ephemeral processing	ONNX on-device + visible UX
UI	Concept	Adaptive states	Control mode + privacy indicators
Wellness	Always-on	Severity-gated	Opt-in preference + severity-gated
Metrics	None	None	Full KPI framework + efficacy loop
Clinical	Disorder modules	DVITE + 8 modules	DVITE + personalization + efficacy

DETECT -> VALIDATE -> INTERVENE -> TRACK -> ESCALATE

Humorix AI v3.0 - Hardened - Adaptive - Clinically-viable